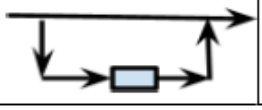
TRABAJO PRACTICO 2

SINTAXIS

Ejercicio 1)

Completar el siguiente cuadro:

Meta símbolos utilizados por		Símbolo utilizado en Diagramas sintácticos	Significado
BNF	**EBNF**		
palabra terminal	palabra terminal		Definición de un elemento terminal
< >	< >	rectángulo	Definición de un elemento no terminal
::=	::=	diagrama con rectángulos, óvalos y flechas	Es un metasímbolo y define una producción / regla
|	(|)	flecha que se divide en dos o más caminos	Selección de una alternativa
< p > < p1 >	{}	flecha que vuelve a la condición	Repetición
	*		Repetición de 0 o más veces
	+		Repetición de 1 o más veces
	[]		Opcional, está presente o no lo está

Ejercicio 2 ¿Cuál es la importancia de la sintaxis para un lenguaje? ¿Cuáles son sus elementos?

La sintaxis es el conjunto de reglas que definen cómo componer letras, dígitos y otros caracteres que permiten formar los programas y que la comunicación entre el programador y la computadora sea clara. Permite escribir programas válidos y coherentes, correctos sintácticamente.

Los elementos de la sintaxis son:

- Alfabeto o conjunto de caracteres:
  - Conjunto finito de símbolos admitido en el lenguaje.
  - La secuencia de bits que compone cada carácter la determina la implementación.
- Identificadores:
  - Cadena de letras y dígitos, que deben comenzar con una letra
  - Si se restringe la longitud se pierde legibilidad
- Operadores:
  - Con los operadores de suma, resta, etc. la mayoria de los leguajes usan +,-. En los otros operadores no hay tanta uniformidad.
- Comentarios y uso de blancos:
  - Hacen los programas mas legibles
- Palabra clave y palabra reservada:
  - Palabra clave: son palabras que tiene un significado dentro de un contexto.
  - Palabra reservada: son palabras claves que además no pueden ser usadas por el programador como identificador de otra entidad.

Ejercicio 3 - Explique a qué se denomina regla lexicográfica y regla sintáctica

Las reglas lexicas son un un conjunto de reglas para formar las “word”, a partir de caracteres del alfabeto.

Las reglas sintacticas son un conjunto de reglas que definen como formar las las “expresiones” y “sentencias”

Ejercicio 4 - En la definición de un lenguaje, a qué se llama palabra reservadas? ¿A qué son equivalentes en la definición de una gramática? De un ejemplo de palabra reservada en el lenguaje que más conoce (Ada,C,Ruby,Python,..)

Las palabras reservadas son palabras claves que no pueden ser utilizadas por el programador para identificar otra entidad, es decir, son palabras con un significado especial dentro del lenguaje y que no pueden ser utilizadas para nombrar variables o funciones. Estas, se corresponden con el conjunto de símbolos terminales de la gramática.
Un ejemplo de palabra reservada en JavaScript es `let`. Esta palabra se utiliza para declarar una variable en JavaScript, por lo que no puede ser utilizada como nombre de variable. Otros ejemplos de palabras reservadas en JavaScript incluyen `if, else, while, for, function, return, true, false, null, undefined, try, catch, throw, typeof,` entre otras.

Ejercicio 5 - Dada la siguiente gramática escrita en BNF

```
G= ( N, T, S, P)
N = {<numero_entero>, <digito> }
T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
S = <numero_entero>
P = {
<numero_entero>::=<digito><numero_entero> | <numero_entero><d
igito> | <digito>
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
}
```

a- Identifique las componentes de la misma

G: Gramatica, son reglas finitas que definen un conjunto infinito de posibles sentencias validas en el leguaje. Formada por la 4-tupla G=(N, T, S, P)

N: Es el conjunto de simbolos no terminales. <numero_entero> y <digito>

T: Es el conjunto de simbolos terminales. En este caso es el conjunto {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

S: es el simbolo distinguido de la gramatica. En este caso <numero_entero>

P: es el conjunto de producciones de la gramatica. En este caso hay dos producciones.

- <numero_entero>::=<digito><numero_entero> | <numero_entero><digito> | <digito>
- <digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

b- Indique porqué es ambigua y corríjala

Una gramática es **ambigua** cuando para una misma cadena se pueden construir **dos o más árboles de derivación diferentes**.
El problema está en esta producción:

```
<numero_entero> ::= <digito><numero_entero> | <numero_entero>
<digito> | <digito>
```

Tiene **dos reglas recursivas**: una que crece por la izquierda y otra por la derecha. Esto crea ambigüedad.

La clave está en el **lado derecho** de cada producción:

- **Regla 1** — el lado izquierdo es `<numero_entero>`, y en el lado derecho también aparece `<numero_entero>` (al final). Se llama a sí misma → **recursiva**.
- **Regla 2** — igual, pero `<numero_entero>` aparece al principio del lado derecho → también **recursiva**.
- **Regla 3** — el lado derecho solo tiene `<digito>`, que es un símbolo diferente. No se llama a sí misma → **no recursiva** (este es el "caso base" que detiene la recursión).

La solución es **eliminar una de las dos recursiones**. Se elige una sola dirección — por convención se usa **recursión por la derecha** (que genera números de izquierda a derecha, como los leemos):

```
<numero_entero> ::= <digito><numero_entero> | <digito>
<digito>        ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Resumen del proceso paso a paso

1. **Identificar la ambigüedad**: buscar producciones que permitan más de un árbol para la misma cadena. En este caso, las dos reglas recursivas (`<digito><NE>` y `<NE><digito>`) compiten entre sí.
2. **Construir dos árboles** para una cadena de ejemplo (como "123") para demostrar concretamente que la gramática es ambigua.
3. **Elegir una sola dirección de recursión** y eliminar la otra. Ambas generan el mismo lenguaje (todos los números enteros no negativos), pero solo una de ellas es no ambigua.
4. **Verificar** que con la gramática corregida, cualquier cadena tenga **un único** árbol de derivación posible.

Ejercicio 6 - Defina en BNF (Gramática de contexto libre desarrollada por Backus- Naur) la gramática para la definición de una palabra cualquiera.

```
G= (N, T, S, P)
N = {<palabra>, <mayuscula>, <minuscula> }
T = {A,..,Z a,..,z}
S = <palabra>
P = {
<palabra>::=<mayuscula><palabra> | <minuscula><palabra> | <mi
nuscula> | <mayuscula>
<mayuscula> ::= A |..| Z
<minuscula> ::= a |..| z
}
```

Ejercicio 7 - Defina en EBNF la gramática para la definición de números reales. Inténtelo desarrollar para BNF y explique las diferencias con la utilización de la gramática EBNF.

```
EBNF G= (N, T, S, P)
N = {<n_real>, <digito>}
T = {0,1,2,3,4,5,6,7,8,9,"+","-",","}
S = <n_real>
P = {
<n_real>::=[-] {<digito>} + [,{<digito>}*]
<digito>::= 0|1|2|3|4|5|6|7|8|9
}
```

```
BNF G= (N, T, S, P)
N = {<n_real>, <integer>, <digito>}
T = {0,1,2,3,4,5,6,7,8,9,"+","-",","}
S = <n_real>
P = {
<n_real>::= <integer> "," <integer> | + <integer> "," <intege
r>  | - <integer> "," <integer> | <integer> | + <integer> | -
<integer>
<integer>::= <digito> | <digito><integer>
```

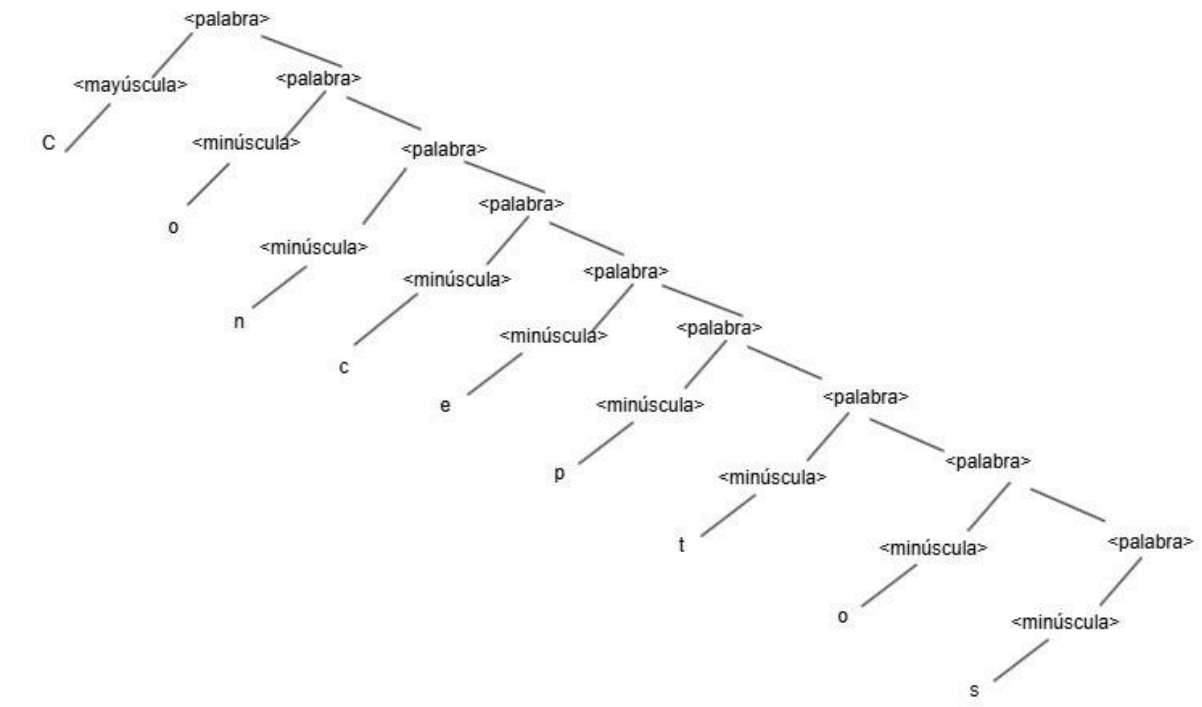
```
<digito>::= 0|1|2|3|4|5|6|7|8|9
}
```

La diferencia más clara entre las dos formas es la cantidad de símbolos no terminales que necesita la BNF para poder definir lo mismo que definimos en la EBNF con menos símbolos, además de la cantidad de producciones

Ejercicio 8 - Utilizando la gramática que desarrolló en los puntos 6 y 7, escriba el árbol sintáctico de:

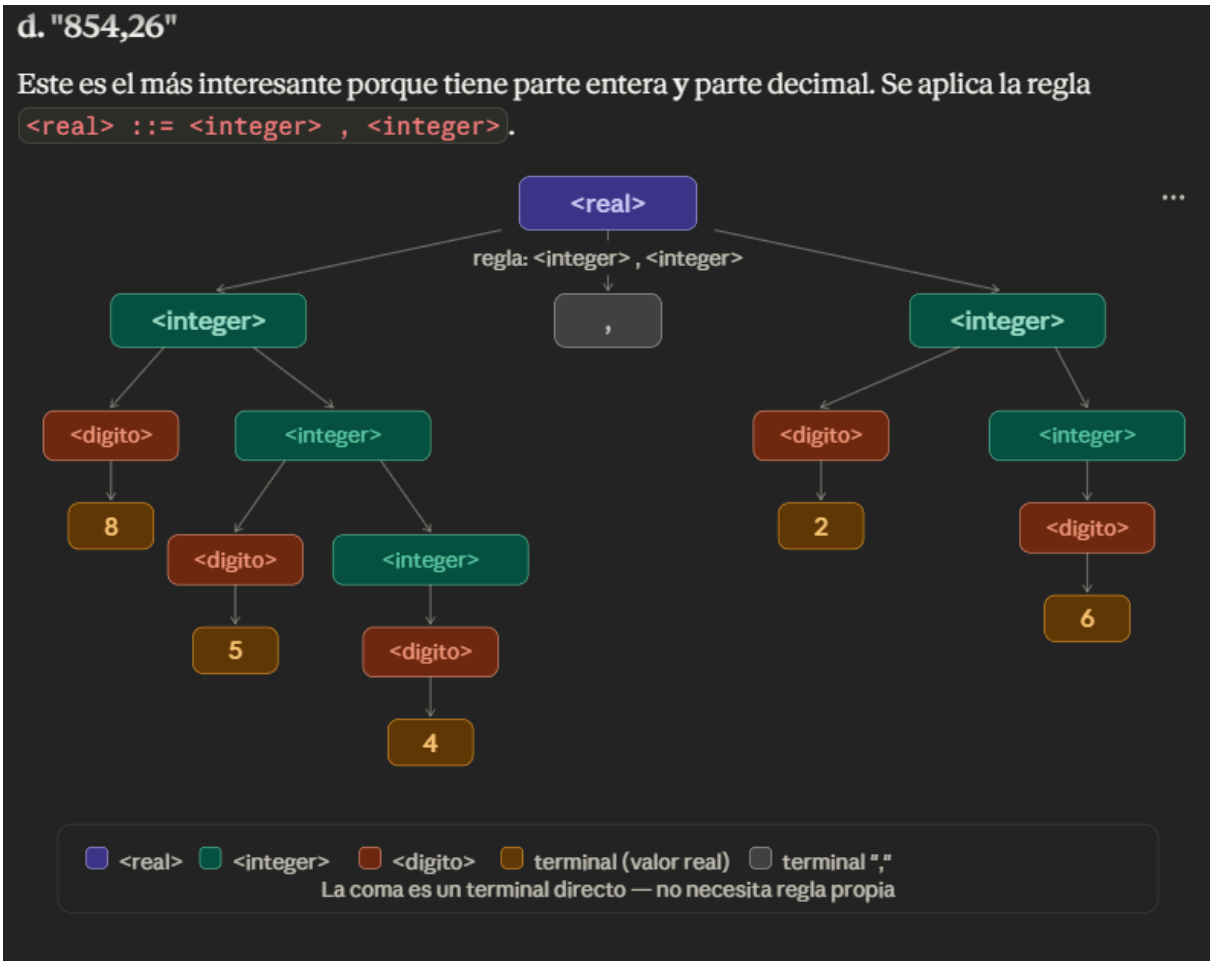
a. Conceptos
b. Programación
c. 1255869
d. 854,26
e. Conceptos de lenguajes

8. a.




```
<real>
└── <integer>
    ├── <digito> → 1
    └── <integer>
        ├── <digito> → 2
        └── <integer>
```

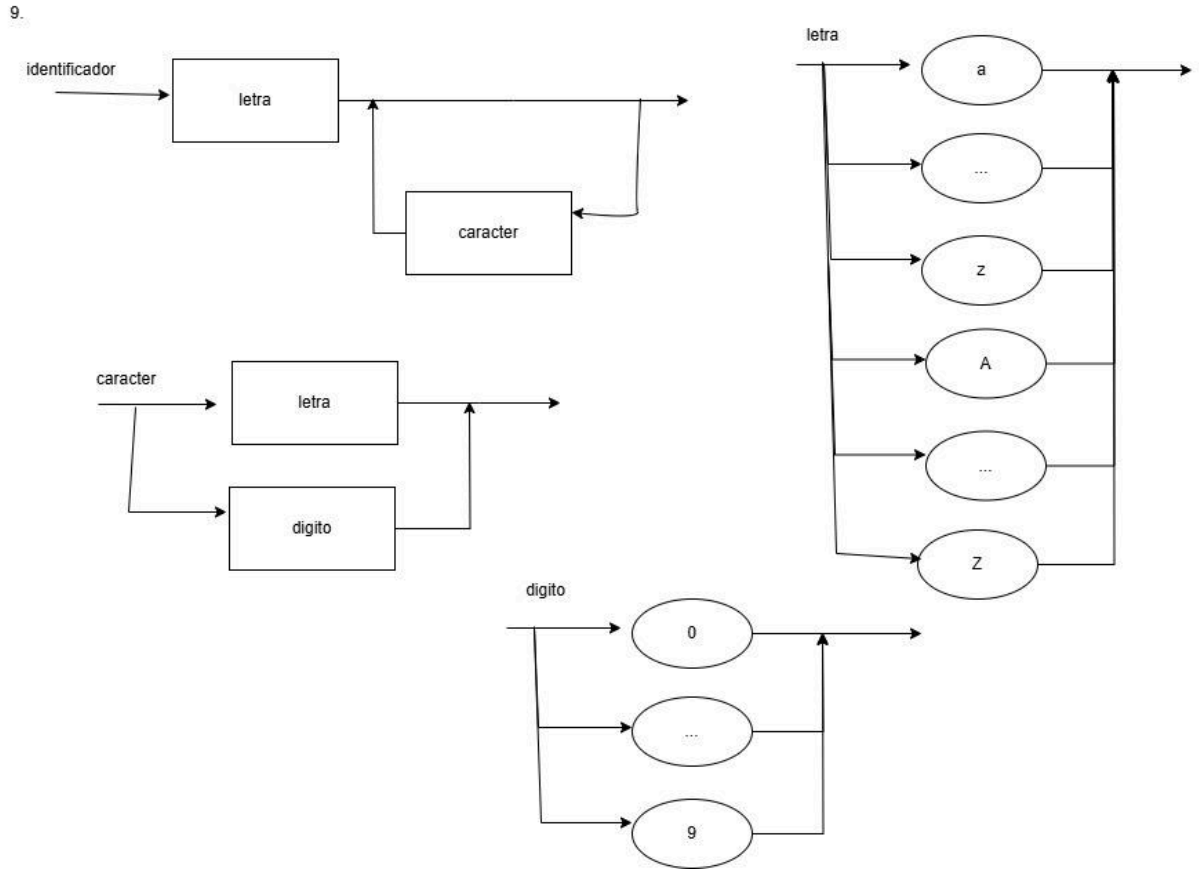
```
            ├── <digito> → 5
            └── <integer>
                ├── <digito> → 5
                └── <integer>
                    ├── <digito> → 8
                    └── <integer>
                        ├── <digito> → 6
                        └── <integer>
                            └── <digito> → 9
```



e. Con las gramáticas definidas no se puede representar una cadena de texto, sino palabras individuales. Podría modificarse la gramática para hacer palabras y agregar " ".

Ejercicio 9 - Defina utilizando diagramas sintácticos la gramática para la definición de un identificador de un lenguaje de programación. Tenga presente como regla que un identificador

no puede comenzar con números.



Ejercicio 10
a) Defina con EBNF la gramática para una expresión numérica, dónde intervienen variables y números. Considerar los operadores +, -, * y / sin orden de prioridad. No considerar el uso de paréntesis.
b) A la gramática definida en el ejercicio anterior agregarle prioridad de operadores.
c) Describa con sus palabras los pasos y decisiones que tomó para agregarle prioridad de operadores al ejercicio anterior.

a)

```
G = {N, T, S, P}
N = {<expr>, <variable>, <operador>, <número>, <dígito>, <let
ra>}
T = {a, ..., z, A, ..., Z, 0, ..., 9, “+”, “-”, “” , “/”}
```

```
S = <expr>
P = { <expr> ::= ( <variable> | <número> ){ <operador>( <vari
able> | <número> )}+
<número> ::= [(+|-)] {<dígito>}+
<variable> ::= <letra>{ ( <letra> | <digito> ) }
<dígito> ::= ( 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 )
<operador> ::= ( + | - | * | / )
<letra> ::= ( a | ... | z | A | ... | Z )
}
```

b)

```
b) A la gramática definida en el ejercicio anterior agregarle
prioridad de operadores.
G = (N, T, S, P)
N = {<exp>, <term>, <elem>, <variable>, <letra>, <num>, <digi
to>}
T = {0..9, a..z, A..Z, +, -, *, /}
S = <exp>
P = { <exp>:: = <term>{(+|-)<term>}*
<term> ::= <elem> { (*|/) <elem> }*
<elem> ::= (<variable> | <num>)
<variable> ::= <letra>{(<letra>  | <digito>)}*
<letra> ::= (a | b | c | …)
<num> ::= <digito>{<digito>}*
<digito> ::= (0 | 1 | 2 | …)
}
```

c - Para agregar prioridad de operadores a la gramática del ejercicio anterior, primero identifiqué que no todos los operadores deben evaluarse de la misma forma: la multiplicación y la división tienen mayor prioridad que la suma y la resta.

A partir de eso, la decisión principal fue **separar la expresión en distintos niveles**, usando nuevos no terminales.

En lugar de tener una única regla para toda la expresión, definí:

- Un nivel superior `<exp>` para manejar las operaciones de suma y resta.
- Un nivel intermedio `<term>` para manejar las operaciones de multiplicación y división.

- Un nivel básico `<elem>` para representar los valores, es decir, números o variables.

Luego, definí las producciones de forma que:

- `<exp>` esté formada por uno o más `<term>` unidos por `+` o .
- `<term>` esté formado por uno o más `<elem>` unidos por o `/`.
- `<elem>` sea un identificador o un número.

De esta manera, la gramática obliga a que primero se formen los `<term>` (resolviendo multiplicaciones y divisiones) y recién después se combinen en `<exp>` mediante sumas y restas.

La prioridad de operadores no se indica explícitamente, sino que se logra gracias a esta **estructura jerárquica de la gramática**, donde los niveles más internos tienen mayor prioridad.

Ejercicio 11: La siguiente gramática intenta describir sintácticamente la sentencia for de ADA, indique cuál/cuáles son los errores justificando la respuesta.

```
N= {<sentencia_for>,<bloque>, <variable>, <letra>, <cadena>,
<digito>, <otro>,
        <operacion>, <llamada_a_funcion>, <numero>,  <sentenc
ia> }
P= { <sentencia_for>::= for (i= IN 1..10) loop <bloque> end l
oop;
<variable>::= <letra> | <cadena>
<cadena>::= { ( <letra> | <digito> | <otro> ) }+
<letra>::=( a | .. | z | A | .. | Z )
<digito>::= ( 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 )
<bloque>::=  <sentencia> | <sentencia> <bloque> | <bloque> <s
entencia> ;
<sentencia>::= <sentencia_asignacion> | <llamada_a_funcion> |
<sentencia_if> |
                                <sentencia_for> |  <sentencia
_while> | <sentencia_switch>
}
```

La gramática presenta varios errores. En primer lugar, no está completamente definida como una 4-tupla, ya que faltan los conjuntos de terminales (T) y el símbolo inicial (S).

Además, existen no terminales declarados pero no definidos mediante producciones, como `<otro>, <operacion>, <llamada_a_funcion> y <numero>`

También hay no terminales utilizados pero no declarados en N, como `<sentencia_asignacion>, <sentencia_if>, <sentencia_while> y <sentencia_switch>`

Por otro lado, la producción de `<bloque>` es ambigua, ya que permite múltiples derivaciones para una misma secuencia de sentencias, y además presenta recursión izquierda, lo que puede generar problemas en el análisis sintáctico.

Finalmente, la producción de `<sentencia_for>` no está correctamente formalizada, ya que mezcla elementos del lenguaje con la notación de la gramática y no define adecuadamente los terminales.

Ejercicio 12 - Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace (https://developer.mozilla.org/es/docs/Web/HTML/Elemento/div)

```
G = (N, T, S, P)
N = { <div>, <elemento_html>, <etiqueta>, <etiquetaCierre>, <
palabra>, <letra> }
T = {A..,Z,a..,z,<,>,/}
s = <div>
P = {
    <letra>::= (A|..|z)
    <palabra>::= {<letra>}+
    <etiqueta>::= "<" <palabra> ">"
    <etiquetaCierre> ::= "</" <palabra> ">"
    <elemento_html> ::= <etiqueta> ( <elemento_html> | <palab
ra> ) <etiquetaCierre>
    <div>::= "<div>"(<elemeto_html> | <palabra>)"</div>"
}
```

Ejercicio 13 - Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace

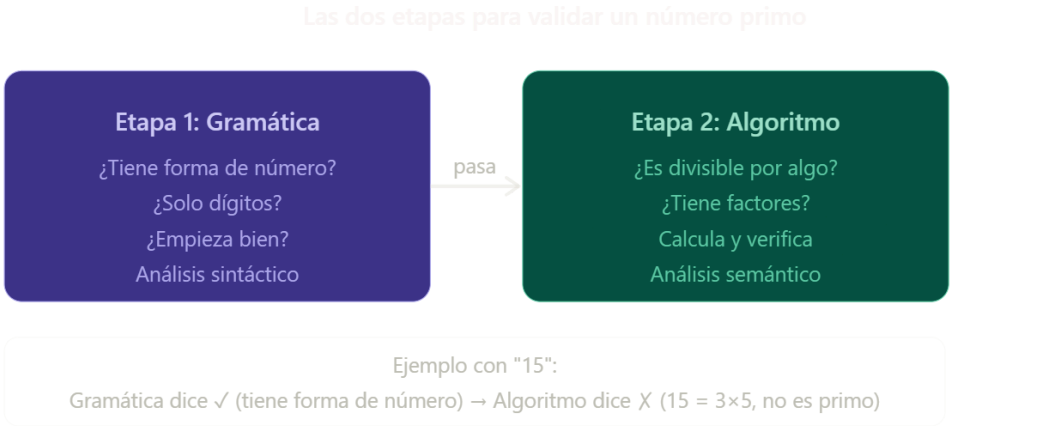
Una gramática solo describe **la forma** de las cadenas. No puede hacer cálculos ni verificar propiedades matemáticas.

Para saber si `7` es primo, necesitás:

1. Tomar el número `7`
2. Dividirlo por `2, 3, 4, 5, 6`
3. Ver si alguna división da exacto
4. Si ninguna da exacto → es primo

Una gramática **no puede dividir números**. No puede hacer ningún cálculo. Solo puede decir *"esta cadena tiene la forma correcta de un número entero"*.

Entonces si escribís `15`, la gramática dice ✓ porque tiene la forma de un número entero. Pero `15` no es primo (`3 × 5`). La gramática no tiene forma de detectar eso.



Entonces la gramática que corresponde usar es la de números enteros

```
G = ( N, T, S, P )
N = { <numero>, <digito> }
T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
```

```
S = <numero>
P = {
  <numero> ::= <digito> { <digito> }*
  <digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
}
```

Esta gramática acepta `7`, `13`, `15`, `100`, `37`... cualquier número entero positivo. No distingue entre primos y no primos porque **no puede hacerlo**.

¿Qué habría que agregar entonces?

La respuesta del ejercicio es exactamente lo que dice: **una etapa semántica o algorítmica**. En la práctica eso significa un programa que recibe el número que pasó la gramática y verifica si es primo:

```
función esPrimo(n):
    si n < 2 → no es primo
    para i desde 2 hasta √n:
        si n es divisible por i → no es primo
    → es primo
```

El concepto importante para llevarte

Esto pasa también en los lenguajes de programación reales. Por ejemplo en Java, la gramática acepta esto:

`int x = "hola";`

Tiene la **forma correcta** de una asignación. Pero el **significado** está mal porque no podés asignar un texto a un entero. Eso lo detecta el compilador en la etapa semántica, no la gramática.

La gramática controla la **forma**. El significado lo controla otra etapa. Son dos cosas separadas.

Ejercicio 14: Sobre un lenguaje de su preferencia escriba en EBNF la gramática para la definición de funciones o métodos o procedimientos (considere los parámetros en caso de ser necesario)

```
G = ( N, T, S, P )

N = { <funcion>, <parametros>, <palabra>, <letra>, <dig> }
```

```
T = { a,...,z,  A,...,Z,  0,...,9 }

S = <funcion>

P = {
  <funcion>    ::= "function" <palabra> <parametros> "{" {<pa
labra>}+ "}"
  <parametros> ::= "(" [ <palabra> { "," <palabra> }* ] ")"
  <palabra>    ::= <letra> { ( <letra> | <dig> ) }*
  <letra>      ::= ( a | ... | z | A | ... | Z )
  <dig>        ::= ( 0 | ... | 9 )
}
```